

Interfaces, Packages, and Multithreaded Programming

Introduction to Interfaces

In Java, an interface is a blueprint of a class. It is a collection of abstract methods (methods without a body) and constant variables. An interface specifies what a class must do, but not how it does it. In other words, it defines a contract that classes can adhere to. Interfaces are a crucial part of achieving abstraction in Java, allowing you to separate the specification of behavior from its implementation.

Key Characteristics of Interfaces:

- **Abstract Methods:** Interfaces primarily contain abstract methods. These methods have no implementation within the interface itself.
- **Constants:** Interfaces can declare constant variables (declared as static and final).
- **Multiple Inheritance:** A class can implement multiple interfaces, allowing it to inherit multiple types of behavior. This addresses the limitations of single inheritance in Java.
- **Abstraction:** Interfaces promote abstraction by hiding the implementation details of methods.
- **Loose Coupling:** means that one class does not need to know the internal details of another class. It interacts with other classes through interfaces, abstract classes, or common behavior, not through concrete implementations..

Defining Interfaces

An interface is defined using the `interface` keyword. The syntax is as follows:

```
interface InterfaceName {  
    // Constant variables (optional)  
    public static final data_type VARIABLE_NAME = value;  
  
    // Abstract methods (without body)  
    public abstract return_type methodName(parameter_list);  
}
```

example:

```
interface Drawable {  
    // Constant variable  
    public static final int DEFAULT_BORDER_WIDTH = 1;  
  
    // Abstract methods  
    public abstract void draw();  
    public abstract void resize(int width, int height);  
}
```

Important Notes:

- All methods declared inside an interface are implicitly `public` and `abstract`. You can explicitly declare them as such, but it's not mandatory.
- All variables declared inside an interface are implicitly `public`, `static`, and `final`.
- Interfaces cannot be instantiated directly. You cannot create an object of an interface.

Extending Interfaces

- An interface can inherit from another interface using the `extends` keyword. This is similar to class inheritance, but with interfaces. When an interface extends another interface, it inherits all the abstract methods and constants of the parent interface.

Syntax:

```
interface ChildInterface extends ParentInterface {  
    // Additional abstract methods and constants  
}
```

Example:

```
interface Shape {  
    void display();  
}  
  
interface Colorable extends Shape {  
    void setColor(String color);  
    String getColor();  
}
```

In this example, the `Colorable` interface extends the `Shape` interface. Any class implementing `Colorable` must provide implementations for both `display()` [from `Shape`] and `setColor()` and `getColor()` [from `Colorable`].

Implementing Interfaces

- A class implements an interface using the `implements` keyword. When a class implements an interface, it must provide concrete implementations for all the abstract methods declared in the interface.

Syntax:

```
class ClassName implements InterfaceName1,  
    InterfaceName2, ... {  
    // Implementations of abstract methods from the  
    // interfaces  
}
```

example:

```
interface Printable {  
    void print();  
}  
  
class Document implements Printable {  
    @Override  
    public void print() {  
        System.out.println("Printing document...");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Document doc = new Document();  
        doc.print(); // Output: Printing document...  
    }  
}
```

Accessing Interface Variables

- Variables declared in an interface are implicitly public, static, and final. This means they are constants and can be accessed directly using the interface name.

```
interface MyInterface {  
    int MAX_VALUE = 100;  
}  
  
class MyClass implements MyInterface {  
    public void displayMaxValue() {  
        System.out.println("Max value: " + MyInterface.MAX_VALUE);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        MyClass obj = new MyClass();  
        obj.displayMaxValue(); // Output: Max value: 100  
        System.out.println("Max value: " + MyInterface.MAX_VALUE); // Accessing  
directly  
    }  
}
```

Introduction to Packages

In Java, a package is a namespace that organizes a set of related classes and interfaces. Think of it as a folder in a file system, where each folder contains related files. Packages serve several important purposes:

- **Organization:** Packages help to structure large projects by grouping related classes together. This makes it easier to find and manage code.
- **Naming Conflicts:** Packages provide a namespace, which prevents naming conflicts between classes with the same name in different packages. For example, you can have a class named `Date` in both the `java.util` package and a custom package without any conflict.

- **Access Control:** Packages provide a mechanism for controlling access to classes and members within a package. You can specify which classes and members are accessible from outside the package.
- **Code Reusability:** Packages promote code reusability by allowing you to easily share and reuse classes across different projects.

```
package mypackage;  
    // defines a package named mypackage  
public class MyClass {  
    public void display() {  
        System.out.println("Hello from  
MyClass in mypackage!");  
    }  
}
```

Types of Packages

Type	Description	Example
Built-in Packages	Provided by Java API	java.util, java.io, java.sql, java.awt, etc.
User-defined Packages	Created by the programmer	package student;; package myproject;

Java API Packages

- The Java API (Application Programming Interface) is a vast collection of pre-written classes and interfaces that provide a wide range of functionalities. These classes are organized into packages, making it easier to find and use the classes you need. Some of the most commonly used Java API packages include:
- `java.lang`: This package contains core classes that are fundamental to the Java language, such as `String`, `Math`, `System`, and `Thread`. This package is automatically imported into every Java program.
- `java.util`: This package contains utility classes, such as `ArrayList`, `HashMap`, `Date`, and `Random`.

- `java.io`: This package contains classes for input and output operations, such as reading from and writing to files.
- `java.net`: This package contains classes for networking operations, such as creating sockets and connecting to servers.
- `java.awt` and `javax.swing`: These packages contain classes for creating graphical user interfaces (GUIs).

Naming Conventions

Packages in Java follow a specific naming convention to ensure uniqueness and avoid conflicts. The naming convention is based on the reverse domain name of the organization or individual creating the package. For example, if an organization's domain name is example.com, the package name would typically start with com.example.

Here are some general guidelines for naming packages:

- Use lowercase letters.
- Use the reverse domain name of your organization or individual.
- Use meaningful names that reflect the purpose of the package.
- Avoid using reserved words as package names.

For example:

- `com.example.myapp.model`
- `org.apache.commons.lang`
- `net.sourceforge.jexcelapi`

Creating Packages

To create a package in Java, you need to declare the package name at the top of your Java source file using the package keyword. For example:

Creating a Package

Syntax:

```
package package_name;
```

Example:

```
package school; // creating package named 'school'
```

```
public class Student {  
    public void show() {  
        System.out.println("Inside Student class of  
school package");  
    }  
}
```

Accessing a Package

There are **three main ways** to access classes from a package:

(a) Using fully qualified name

```
public class Test {  
    public static void main(String[] args) {  
        school.Student s = new school.Student();  
        s.show();  
    }  
}
```

(b) Using import package.class

```
import school.Student;  
public class Test {  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.show();  
    }  
}
```

(c) Using import package.*

```
import school.*;
public class Test {
    public static void main(String[] args) {
        Student s = new Student();
        s.show();
    }
}
```

Using a Package

```
import java.util.Scanner;

public class Example {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("Enter name:");
        String name = input.nextLine();
        System.out.println("Hello, " + name);
    }
}
```

Advantages of Using Packages

- Avoids naming conflicts
- Provides access control
- Makes code modular and maintainable
- Encourages code reuse

Adding a Class to a Package

```
package com.mycompany.shapes;
```

```
public class Circle {  
    public void draw() {  
        System.out.println("Drawing a circle");  
    }  
}
```

```
com/  
mycompany/  
shapes/  
Circle.java
```

Using the Class from Another Package

In another file (outside the package), you can import and use it

```
import com.mycompany.shapes.Circle;
```

```
public class Test {  
    public static void main(String[] args) {  
        Circle c = new Circle();  
        c.draw();  
    }  
}
```

Hiding Classes (Encapsulation and Access Control)

Modifier	Accessibility	Description
public	Accessible from everywhere	The class can be used by any other class
default (no modifier)	Accessible only within the same package	The class is hidden from other packages

Example:

Hiding Classes in a Package

// File: com/mycompany/internal/Helper.java

```
package com.mycompany.internal;
```

// Default access modifier – class is package-private

```
class Helper {
```

```
    void showMessage() {
```

```
        System.out.println("Internal Helper");
```

```
    }
```

```
}
```

Sometimes you want certain classes in a package to be used internally and hidden from other packages.

```
// File: com/mycompany/publicapi/MainClass.java  
package com.mycompany.publicapi;
```

```
import com.mycompany.internal.Helper; // ✕ Error –  
Helper is not public
```

```
public class MainClass {  
    public static void main(String[] args) {  
        Helper h = new Helper(); // Not accessible  
        h.showMessage();  
    }  
}
```

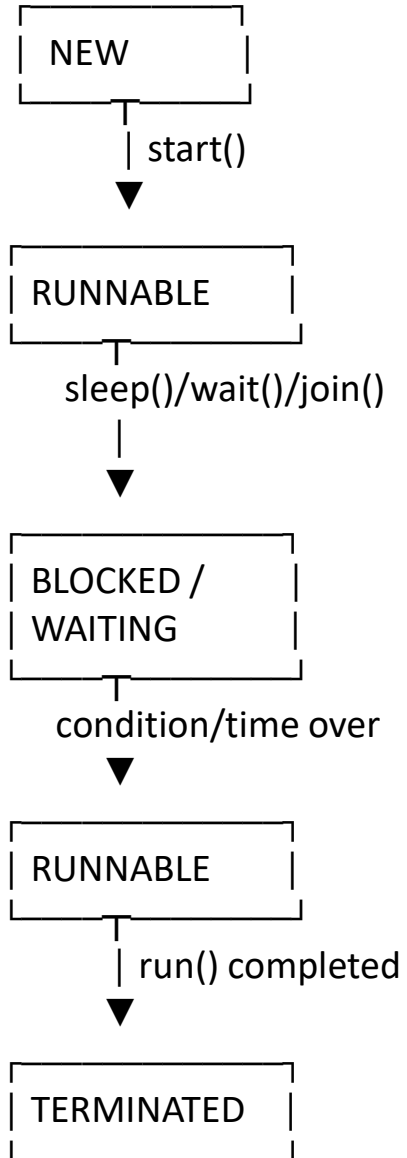
Introduction to Threads

- Threads are lightweight processes that allow concurrent execution of multiple parts of a program.
- Java, multithreading allows you to run two or more parts of a program concurrently. Each part that runs simultaneously is called a **thread**.
- They are essential for building responsive and efficient applications, especially in scenarios involving I/O operations, complex calculations, or user interface updates.
- In Java, threads are managed by the Java Virtual Machine (JVM), which provides a platform-independent way to create and control them.

Life Cycle of a Thread

Every thread in Java passes through several **states** during its execution.

State	Description	Example
New	Thread is created but not yet started.	Thread t = new Thread();
Runnable	Thread is ready to run or running.	t.start();
Blocked	Waiting to acquire a monitor lock (e.g., synchronized).	Enters a locked section
Waiting	Waiting indefinitely for another thread.	wait()
Timed Waiting	Waiting for a specified period.	sleep(ms), join(ms)
Terminated	Thread has completed execution.	run() finished



```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class ThreadLifeCycleSimple {
    public static void main(String[] args) {
        MyThread t1 = new MyThread(); // Thread created → NEW state
        System.out.println("Thread state after creation: " + t1.getState());

        t1.start(); // Thread started → RUNNABLE state
        System.out.println("Thread state after start(): " + t1.getState());

        try {
            Thread.sleep(100); // Give time for t1 to finish
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        System.out.println("Thread state after completion: " + t1.getState());
    }
}
```

Creating Threads

There are two common ways to create a thread:

1. Extending the Thread class
2. Implementing the Runnable interface

1.Extending the Thread class

- **Thread** is a built-in class that represents a **thread of execution**.
- By **extending Thread**, you can **create your own thread class**.
- You **override the run() method** to define the task that the thread will perform.
- Then you call **start()** on the thread object to run it concurrently with other threads.

// Simple thread example

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Hello from the thread!");  
    }  
}
```

```
public class SimpleThread {  
    public static void main(String[] args) {  
        MyThread t = new MyThread(); // Create thread  
        t.start();                    // Start thread  
  
        System.out.println("Hello from main!");  
    }  
}
```

2. Implementing the Runnable interface

This is the **simplest way** to create a thread with Runnable in Java.

```
public class SimpleRunnable {  
    public static void main(String[] args) {  
        // Create a Runnable using a lambda (simplest way)  
        Runnable task = () -> {  
            System.out.println("Hello from the thread!");  
        };  
  
        // Pass Runnable to Thread and start it  
        Thread t = new Thread(task);  
        t.start();  
  
        // Main thread message  
        System.out.println("Hello from main!");  
    }  
}
```

Stopping and Blocking a thread

Blocking a Thread (using sleep)

When a thread calls `Thread.sleep(time)`, it **pauses (blocks)** for the given time and allows other threads to run

```
class BlockingExample extends Thread {  
    public void run() {  
        for (int i = 1; i <= 3; i++) {  
            System.out.println(getName() + " - Step " + i);  
            try {  
                Thread.sleep(1000); // block (pause) for 1 second  
            } catch (InterruptedException e) {  
                System.out.println(getName() + " interrupted!");  
            }  
        }  
    }  
}  
  
public static void main(String[] args) {  
    BlockingExample t1 = new BlockingExample();  
    t1.setName("Worker-1");  
    t1.start();  
}
```

OUTPUT

Worker-1 - Step 1

Worker-1 - Step 2

Worker-1 - Step 3

Thread Priority

In Java, **thread priority** is used to tell the **CPU** which thread should be given **more importance**

- Each thread has a **priority value** — a number between **1 and 10**.

Constant	Value	Meaning
Thread.MIN_PRIORITY	1	Lowest priority
Thread.NORM_PRIORITY	5	Normal (default) priority
Thread.MAX_PRIORITY	10	Highest priority

- When multiple threads are **ready to run**, the **Java scheduler** (part of JVM) **may** give more CPU time to higher-priority threads.
- it **depends on the operating system's scheduler** too.

So, a higher priority **does not guarantee** it will always run first.

```
class PriorityDemo extends Thread {  
    public void run() {  
        System.out.println(getName() + " running with  
priority: " + getPriority());  
    }  
}
```

```
public class ThreadPriorityExample {  
    public static void main(String[] args) {  
        PriorityDemo t1 = new PriorityDemo();  
        PriorityDemo t2 = new PriorityDemo();  
        PriorityDemo t3 = new PriorityDemo();  
    }  
}
```

```
t1.setName("LowPriority");
    t2.setName("NormalPriority");
    t3.setName("HighPriority");

    t1.setPriority(Thread.MIN_PRIORITY); // 1
    t2.setPriority(Thread.NORM_PRIORITY); // 5 (default)
    t3.setPriority(Thread.MAX_PRIORITY); // 10

    t1.start();
    t2.start();
    t3.start();
}
}
```

OUTPUT:

LowPriority running with priority: 1

HighPriority running with priority: 10

NormalPriority running with priority: 5

Thread Methods in Java

Threads in Java are used to perform multiple tasks **at the same time**.

The **Thread class** provides several methods to **control the thread's behavior** — like starting, sleeping, pausing, or checking status.

Commonly Used Thread Methods

Method	Description
start()	Starts a new thread and calls the run() method automatically.
run()	Contains the code that executes when the thread runs.
sleep(ms)	Stops (pauses) the thread for a specific time in milliseconds.
join()	Makes one thread wait for another to complete.
setName(String name)	Sets the thread's name.
getName()	Returns the thread's name.
setPriority(int p)	Sets the thread's priority (1–10).
getPriority()	Returns the thread's priority.
isAlive()	Returns true if the thread is still running.
currentThread()	Returns a reference to the currently running thread.

Simple Example — Using Thread Methods

```
class MyThread extends Thread {  
    public void run() {  
        for (int i = 1; i <= 3; i++) {  
            System.out.println(getName() + " is running: " + i);  
            try {  
                Thread.sleep(500); // pause for 0.5 seconds  
            } catch (InterruptedException e) {  
                System.out.println(getName() + " interrupted!");  
            }  
        }  
    }  
}
```

```
public class ThreadMethods {  
    public static void main(String[] args) throws InterruptedException {  
        MyThread t1 = new MyThread();  
        MyThread t2 = new MyThread();  
  
        t1.setName("Thread-1");    // set name  
        t2.setName("Thread-2");    // set name  
  
        t1.setPriority(8);          // set priority  
        t2.setPriority(4);          // set priority  
  
        System.out.println("Starting threads...");  
        t1.start();    // start first thread  
        t2.start();    // start second thread  
  
        t1.join();    // main waits for t1  
        t2.join();    // then waits for t2  
  
        System.out.println("Main thread finished after both threads.");  
    }  
}
```

Output

Starting threads...

Thread-1 is running: 1

Thread-2 is running: 1

Thread-1 is running: 2

Thread-2 is running: 2

Thread-1 is running: 3

Thread-2 is running: 3

Main thread finished after both threads.

Thread Exceptions in Java

When working with **threads**, some **errors or problems (exceptions)** can occur during thread execution.

These are called **Thread Exceptions**.

Exception Name	When It Occurs	Example / Explanation
InterruptedException	When a sleeping or waiting thread is interrupted by another thread.	Happens when a thread is in sleep() or join() and another thread calls interrupt().
IllegalThreadStateException	When a thread is started more than once.	Calling start() twice on the same thread object.
SecurityException	When the current thread tries to perform an operation it's not allowed to do.	Example: setting another thread's priority without permission.
NullPointerException	When a null thread reference is used.	Example: calling t.start() when t is null.

Simple Example 1 — InterruptedException

```
class MyThread extends Thread {  
    public void run() {  
        try {  
            System.out.println("Thread is sleeping...");  
            Thread.sleep(2000); // sleep for 2 seconds  
            System.out.println("Thread woke up!");  
        } catch (InterruptedException e) {  
            System.out.println("Thread was interrupted!");  
        }  
    }  
}
```

```
public class ThreadExceptionDemo {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start();  
        t.interrupt(); // interrupt the sleeping thread  
    }  
}
```

Output:

Thread is sleeping...

Thread was interrupted!

Simple Example 2 — IllegalStateException

```
class Demo extends Thread {  
    public void run() {  
        System.out.println("Thread is running...");  
    }  
}  
  
public class IllegalStateDemo {  
    public static void main(String[] args) {  
        Demo t = new Demo();  
        t.start();  
        t.start(); // ✗ Error: cannot start thread twice  
    }  
}
```

Output:

Exception in thread "main" java.lang.IllegalThreadStateException

Synchronization

- When **two or more threads** try to access the **same data or object**, problems can occur.
- To prevent this, we use **synchronization**.
- It means **only one thread at a time** can use a shared resource.
- Synchronization means **one thread at a time** can use a shared resource (like a method or variable).
- It helps to **avoid confusion or wrong results** when many threads run together.

```
class BankAccount {  
    int balance = 1000;  
  
    // synchronized method to deposit money  
    synchronized void deposit(int amount) {  
        balance = balance + amount;  
        System.out.println(Thread.currentThread().getName() +  
            " deposited " + amount + ", Balance: " + balance);  
    }  
}
```

```
class MyThread extends Thread {  
    BankAccount account;  
  
    MyThread(BankAccount account, String name) {  
        super(name);  
        this.account = account;  
    }  
}
```

```
public void run() {  
    account.deposit(500);  
}  
}
```

```
public class SyncExampleSimple {  
    public static void main(String[] args) {  
        BankAccount acc = new BankAccount(); // shared account  
        object  
  
        MyThread t1 = new MyThread(acc, "Person 1");  
        MyThread t2 = new MyThread(acc, "Person 2");  
  
        t1.start();  
        t2.start();  
    }  
}
```